

BACKTRACK 2



Good

Evening

Will start at

9:06 pm

*

Content

01. Print paths in staircase
02. Print all paths from src to dest
03. shortest path in binary maze

Assignment | No of questions:

0 / 3 Attempted

[ALL QUESTIONS](#)

S1 > Q1 Q2 Q3

S1: Default Section

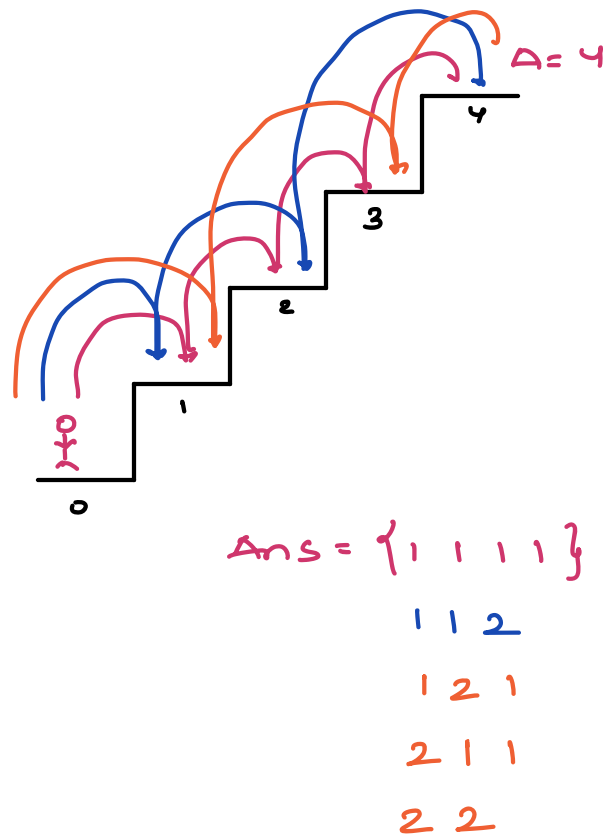
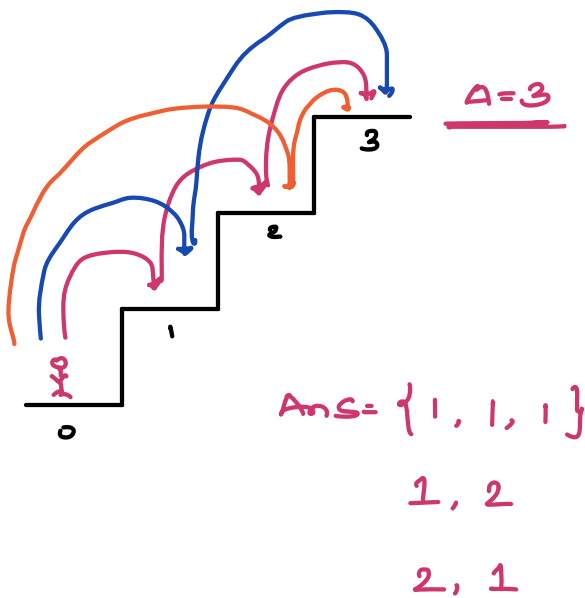
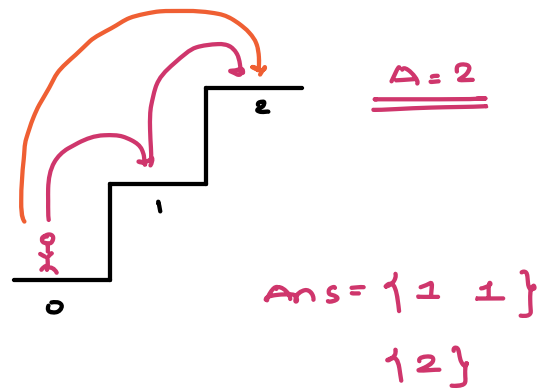
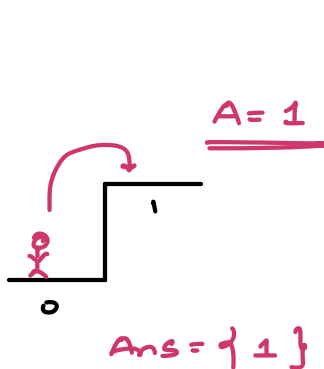
#	QUESTION	DIFFICULTY	TYPE	STATUS	ACTION	BATCH PSP	OVERALL PSP
1	Permutations	Medium	</> Code	Not Attempted	Solve	8.39	9.18
2	Generate all Parentheses II	Medium	</> Code	Not Attempted	Solve	9.68	10.2
3	Generate Subsets	Medium	</> Code	Not Attempted	Solve	9.03	8.5

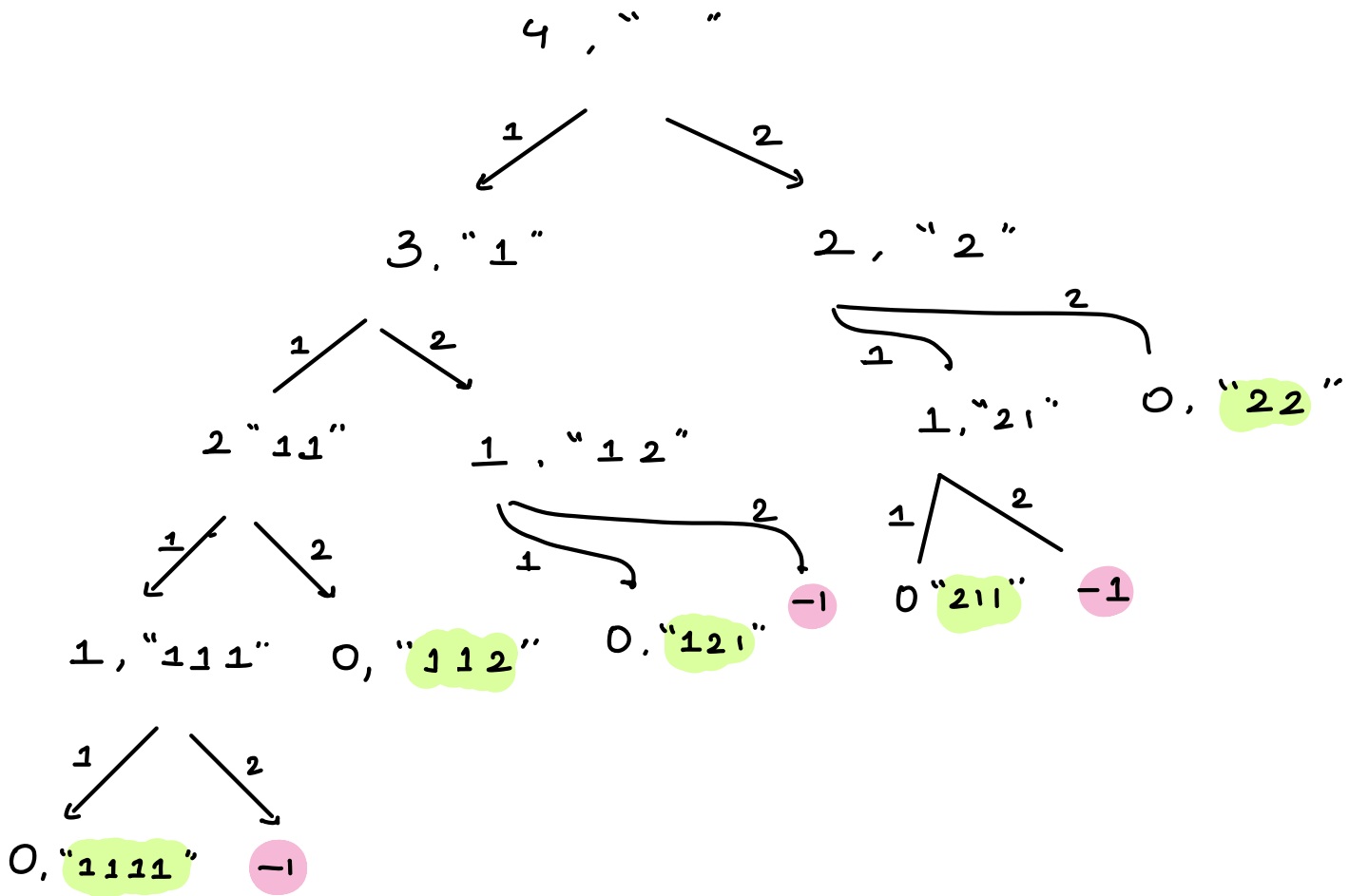
* Print all paths in Staircase

You are climbing a staircase & it takes A steps to reach top.

Each time we can go climb 1 step or 2 step.

How many distinct ways to climb at the top





9:29 PM → 9:31 PM

```

public class Solution {
    public ArrayList<ArrayList<Integer>> WaysToClimb(int A) {
        ArrayList<ArrayList<Integer>>ans=new ArrayList<>();
        ArrayList<Integer>small=new ArrayList<>();

        ways(A,small,ans);
        return ans;
    }
    public void ways(int A,ArrayList<Integer>small, ArrayList<ArrayList<Integer>>ans ){
        if(A<0) return;
        if(A==0){
            ans.add(new ArrayList<>(small));
            return;
        }

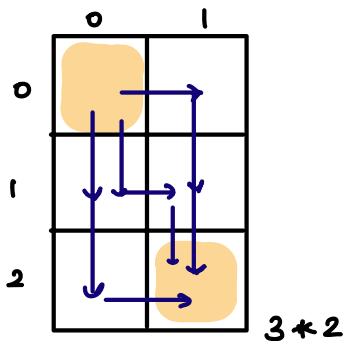
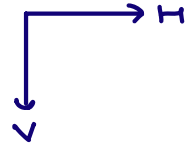
        small.add(1);
        ways(A-1,small,ans);
        small.remove(small.size()-1);

        small.add(2);
        ways(A-2,small,ans);
        small.remove(small.size()-1);
    }
}

```

* Print Maze path

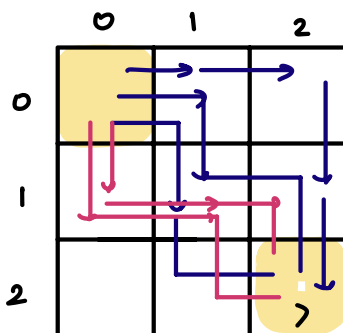
We will be given src & destination. Print all paths from src to destination



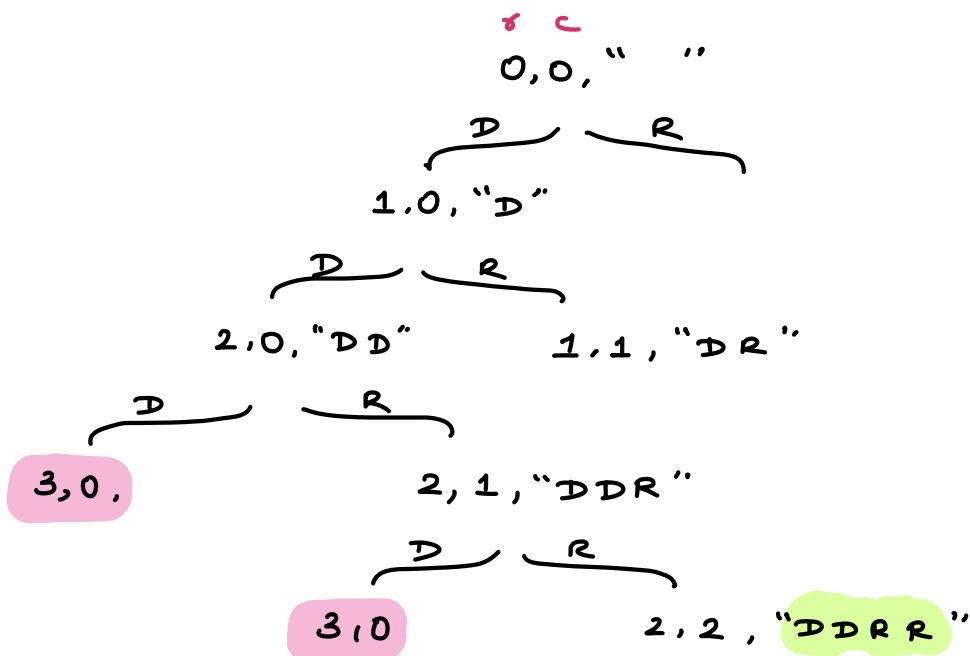
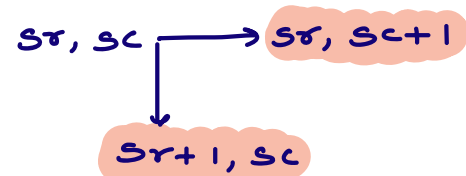
src = (0,0)

dest = (2,1)

HVV
VVH
VHV



HHVV
HVHV
HVVH
VHHV
VHVH
VVHH



printpath (int sr, sc, dr, dc, str, AL<string> ans) {

if (sr == dr && sc == dc) {

ans.add(str);

return;

if (sr < dr) {

printpath (sr+1, sc, dr, dc, str+"D", ans);

if (sc < dc) {

printpath (sr, sc+1, dr, dc, str+"R", ans);

10:08 → 10:18 pm

DSA 4.2

→ Backtracking

→ NQueen

→ Sudoku

→ Word break

Competitive programming

→ 2 Graphs

→ DP on strings

→ 2 Tries

→ Inverse modulo

Problem Statement

Given an $M \times N$ matrix where each element can either be 0 or 1. We need to find the length of **shortest path** between a given **source** cell to a **destination** cell.

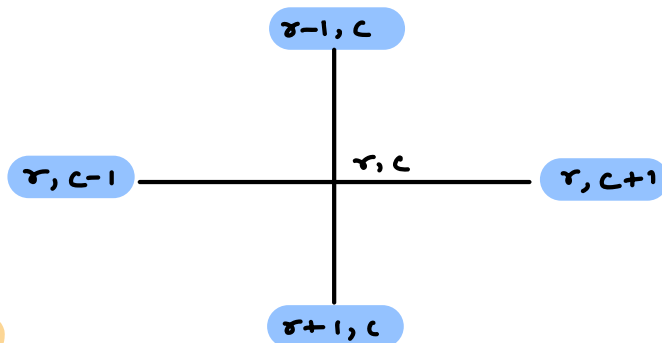
A cell with value 0 denotes that it's a hurdle. The path can only be created out of the cells with values 1.

If **NO** path exists then print -1.

1	1	0	0
0	1	1	0
0	1	1	1
0	1	1	1

$(0,0)$ to $(3,3)$

Ans = 6



	0	1	2
0	1	1	1
1	1	0	1
2	1	1	1

$(0,0)$ to $(0,2)$

Ans = 2

1	0	1
1	0	1
1	0	1

$(0,0)$ to $(0,2)$

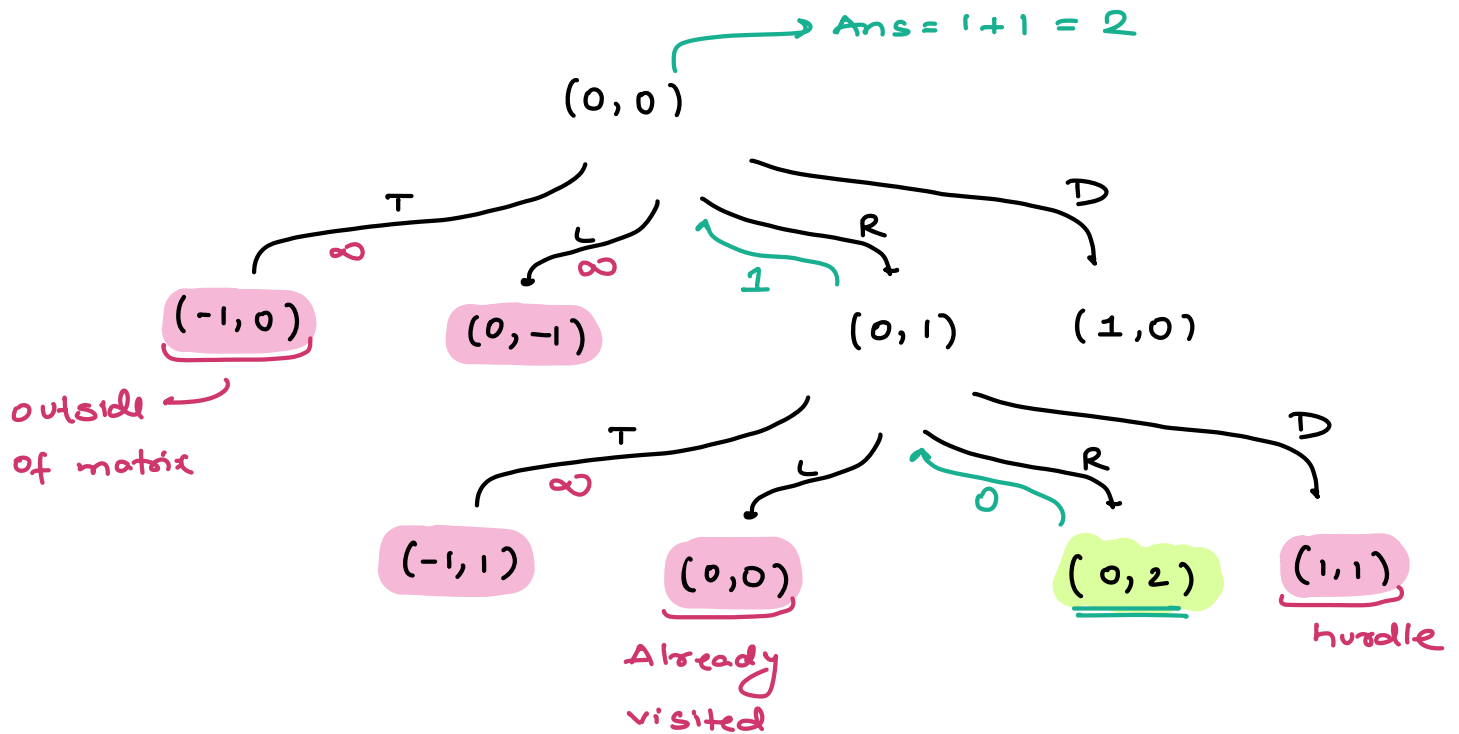
Ans = -1

	0	1	2
0	1	1	1
1	1	0	1
2	1	1	1

(0,0) to (0,2)

	0	1	2
0	T	T	T
1	F	F	F
2	F	F	F

← r, c → r, c+1



$$\text{Dir}[] = \begin{bmatrix} -1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & -1 \end{bmatrix}$$

```
for (d=0; d<4; d++) {
```

```
    int nr = sr + Dir[d][0]
```

```
    int nc = sc + Dir[d][1]
```

```
}
```

```

int findshortest (int [][]A, sr, sc, dr, dc, vis[][] )

if (sr == dr && sc == dc) return 0;

vis[sr][sc] = true;

min = ∞

for (d = 0; d < 4; d++) {
    int nr = sr + Dir[d][0]
    int nc = sc + Dir[d][1]

    if (nr ≥ 0 && nc ≥ 0 && nr < n && nc < m &&
        A[nr][nc] == 1 && vis[nr][nc] == false)
        int x = findshortest (A, nr, nc, dr, dc, vis);
        min = Math.min (min, x)
    }
}
vis[sr][sc] = false;

return min == ∞ ? ∞ : min + 1
}

```

$TC: O(4^{n+n})$
 $SC: O(n+n)$

solve () {

boolean [][] vis = new boolean[n][m];

int ans = findshortest (A, sr, sc, dr, dc , vis)

return ans == ∞ ? -1 : ans;

}

```
public class Solution {
    int[][] dir={{-1,0},{0,1},{1,0},{0,-1}};
    public int FindShortestPath(int[][] A, int sr, int sc, int dr, int dc) {
        int n=A.length;
        int m=A[0].length;
        // boolean[][] vis=new boolean[n][m];

        int ans=findshortest(A,sr,sc,dr,dc,new boolean[n][m]);
        return ans==Integer.MAX_VALUE?-1:ans;
    }
    public int findshortest(int [][]A,int sr,int sc,int dr,int dc,boolean[][]vis){
        int n=A.length;
        int m=A[0].length;

        if(sr==dr && sc==dc) return 0;
        vis[sr][sc]=true;
        int min=Integer.MAX_VALUE;

        for(int d=0;d<4;d++){
            int nr=sr+dir[d][0];
            int nc=sc+dir[d][1];

            if(nr>=0 && nr<n && nc>=0 && nc<m && A[nr][nc]==1 && vis[nr][nc]==false){
                min=Math.min(min,findshortest(A,nr,nc,dr,dc,vis));
            }
        }

        vis[sr][sc]=false;
        return min==Integer.MAX_VALUE? min: min+1;
    }
}
```